敏捷開發方法: Scrum

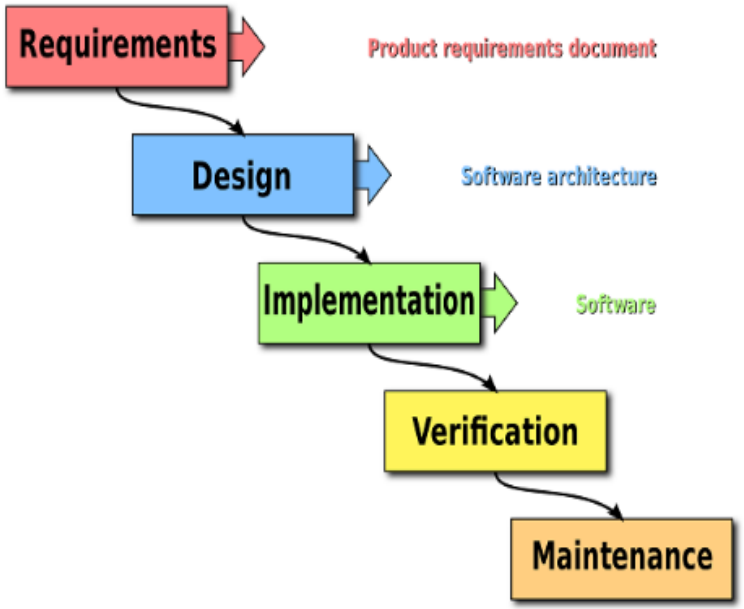
海大資工 馬尚彬

請註冊與觀看課程：
https://www.openedu.tw/course?id=1619

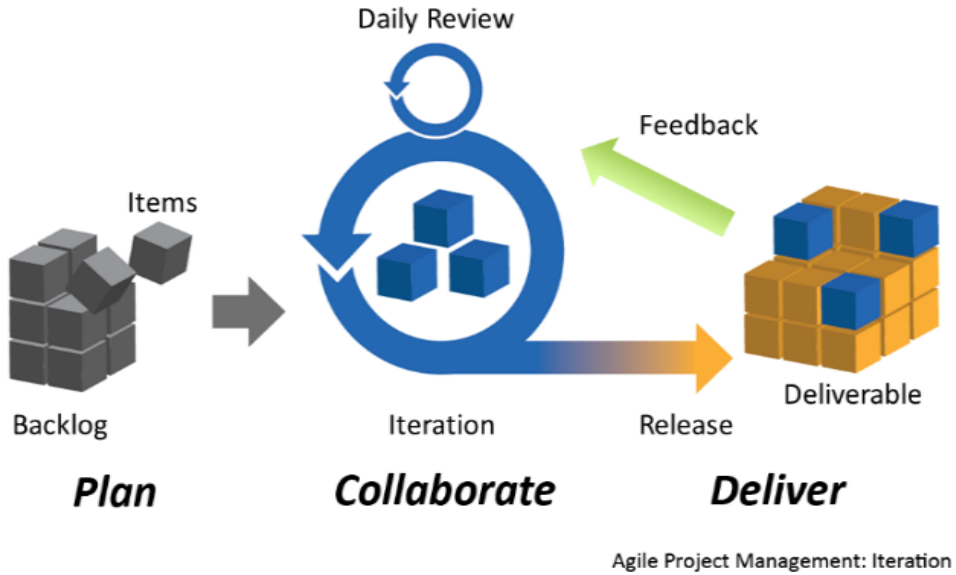
單元大綱

- 敏捷概念與思維
- Scrum流程框架簡介
- Scrum產出物
- Scrum流程活動
- Scrum參與角色
- Scrum的理論與價值觀
- 補充資料: 極限製程(XP)

著名的軟體開發模式



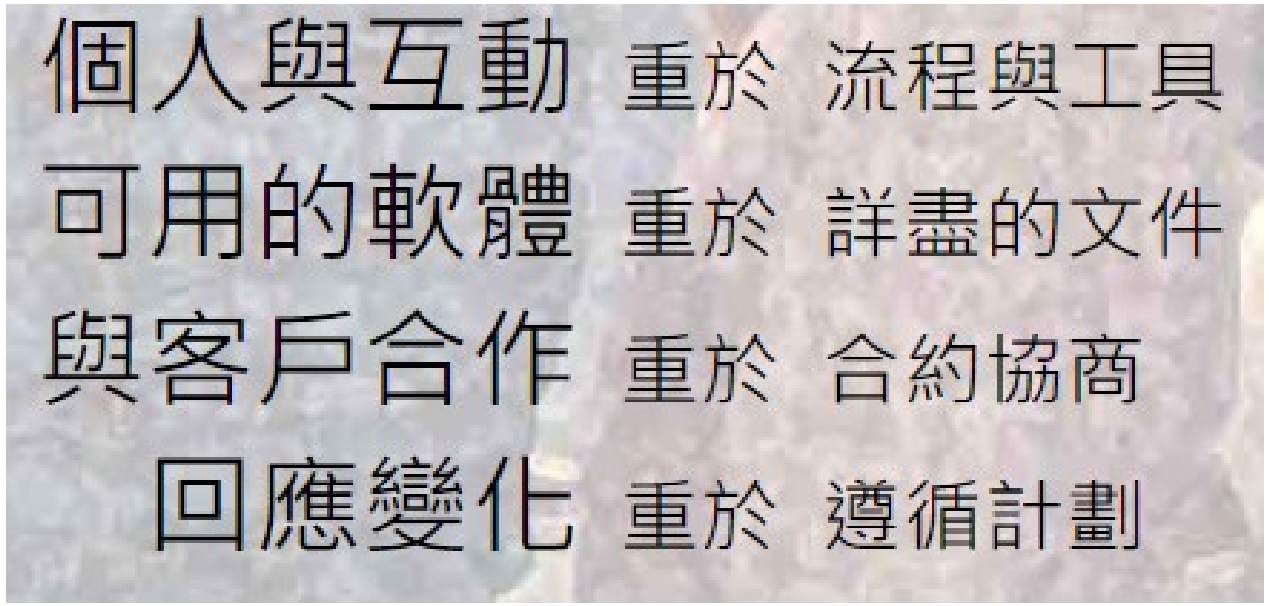
瀑布模型
(Waterfall Model)



敏捷開發方法
(Agile Method)

請問瀑布模型特性是甚麼？會有甚麼侷限？

敏捷軟體開發宣言



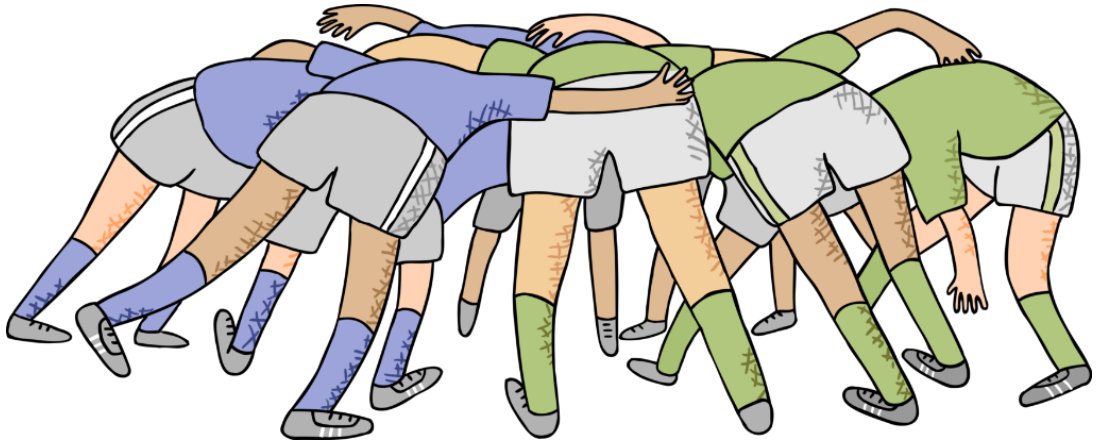
https://agilemanifesto.org/iso/zhcht/manifesto.html

敏捷是一種理念和思考軟體開發的方式，
我們不能說「做」敏捷或「使用」敏捷。

- 以學生團隊的情境來解讀：
  - 我們需要工具與流程來開發軟體，但人與人間的溝通更為重要。
  - 我們需要文件來記錄重要階段的成果，但開發出能運作的軟體功能更為重要。
  - 我們要跟需求端/使用端充分合作，一起發想、製作出好軟體。
  - 我們需要訂計畫，但若有變化可調整計畫。

敏捷(Agile)與Scrum

- Scrum原始意義是橄欖球運動中的列陣爭球。
  - https://www.world.rugby/the-game/beginners-guide/scrum
- 目前大家熟知的Scrum是一種敏捷方法(Agile Method)
  - 提供敏捷的實務做法。
  - 適用於軟體開發之協作。
  - 會頻繁產出實際成果，並累積成果。



https://www.wannapik.com/vectors/16736

敏捷(Agile)思維1

- 定義式(理論)流程
  - 指令與控制
  - 規劃期望發生甚麼
  - 強調**計畫**
  - 應用變更控制機制
- 實務流程
  - 根據歷程來學習
  - 為改變(需求、環境)而規劃
  - **擁抱改變**
  - 觀察並調適

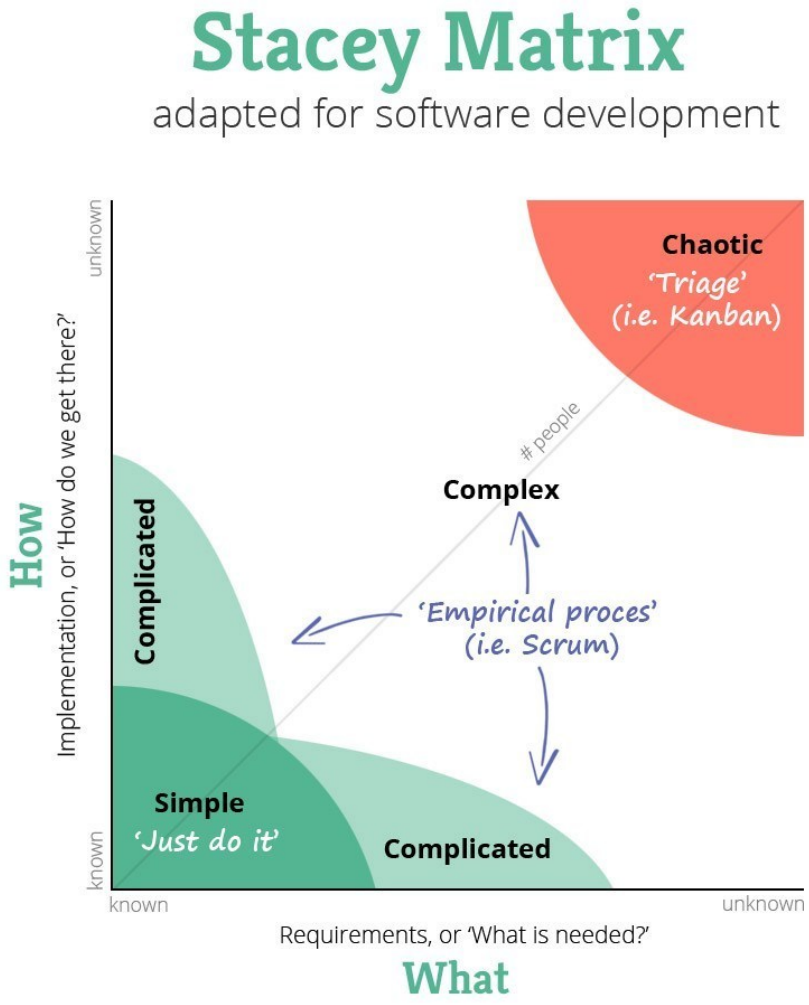
敏捷(Agile)思維2

- 試想以下情境：兩人一組，一人當老闆，一人當員工
  - 方案1：
    - 老闆負責在一段時間內給員工精確指令
      - 左轉、右轉、向後轉、前進N步
    - 員工負責遵守老闆指令
  - 方案2：
    - 員工負責以自己的方式，在一半時間內達成目標
    - 老闆負責給予員工必要協助
- 哪一組的績效會比較好(走的步數比較多)？

敏捷(Agile)思維3

- 當程序與操作很容易理解，使用定義式程序是適合的。
- 當程序較複雜或無法預期，使用經驗式方法是適當的選擇。

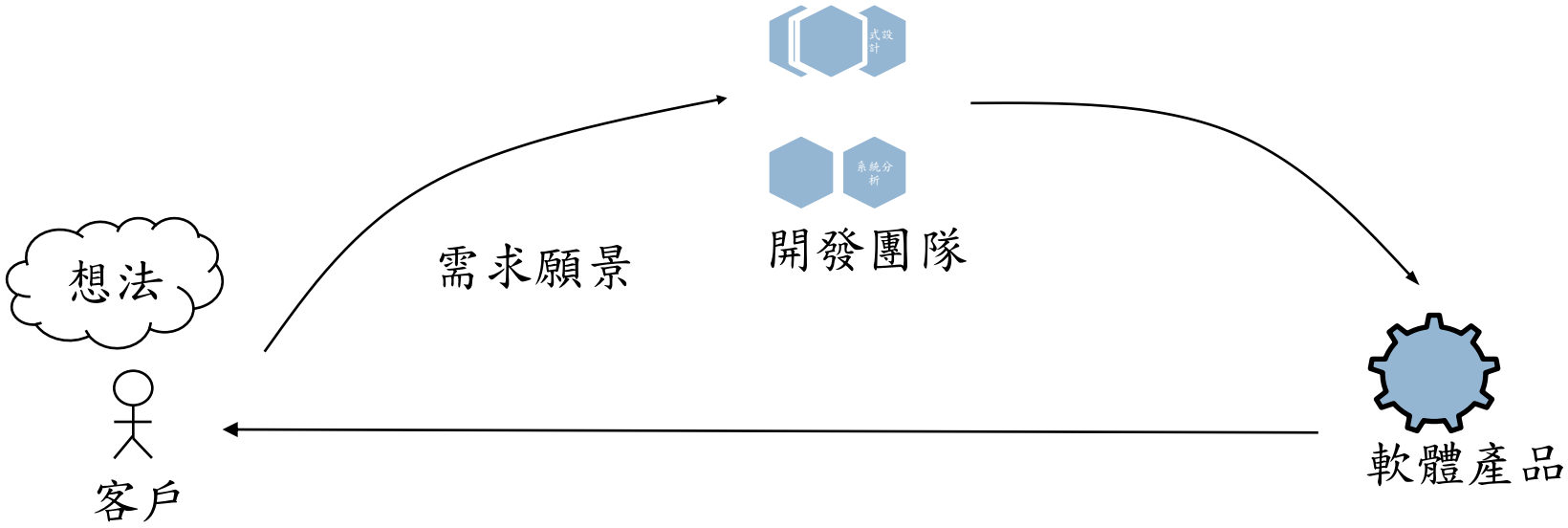
敏捷(Agile)思維3



- 專案根據How和What兩個面向，可分為Simple、Complicated、Complex、Chaotic四種複雜程度。
- 傳統瀑布式模型較適合因應Simple專案。
- 面對Complicated與Complex專案，需要更靈活的作法。

敏捷(Agile)思維4

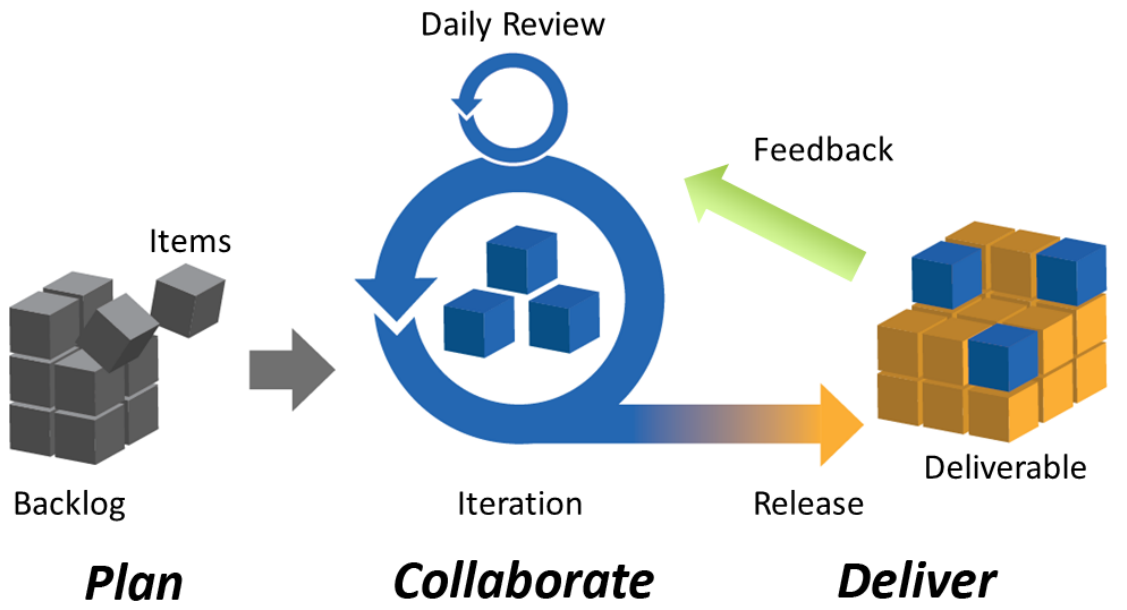
- 軟體開發
  - 將需求轉化成軟體產品，並安裝於預計環境後可如預期執行，即可實現需求願景。



  - 若以**迭代(Iterative)**的方式、**遞增式(Incremental)**地進行開發，較容易逐步實現需求願景。

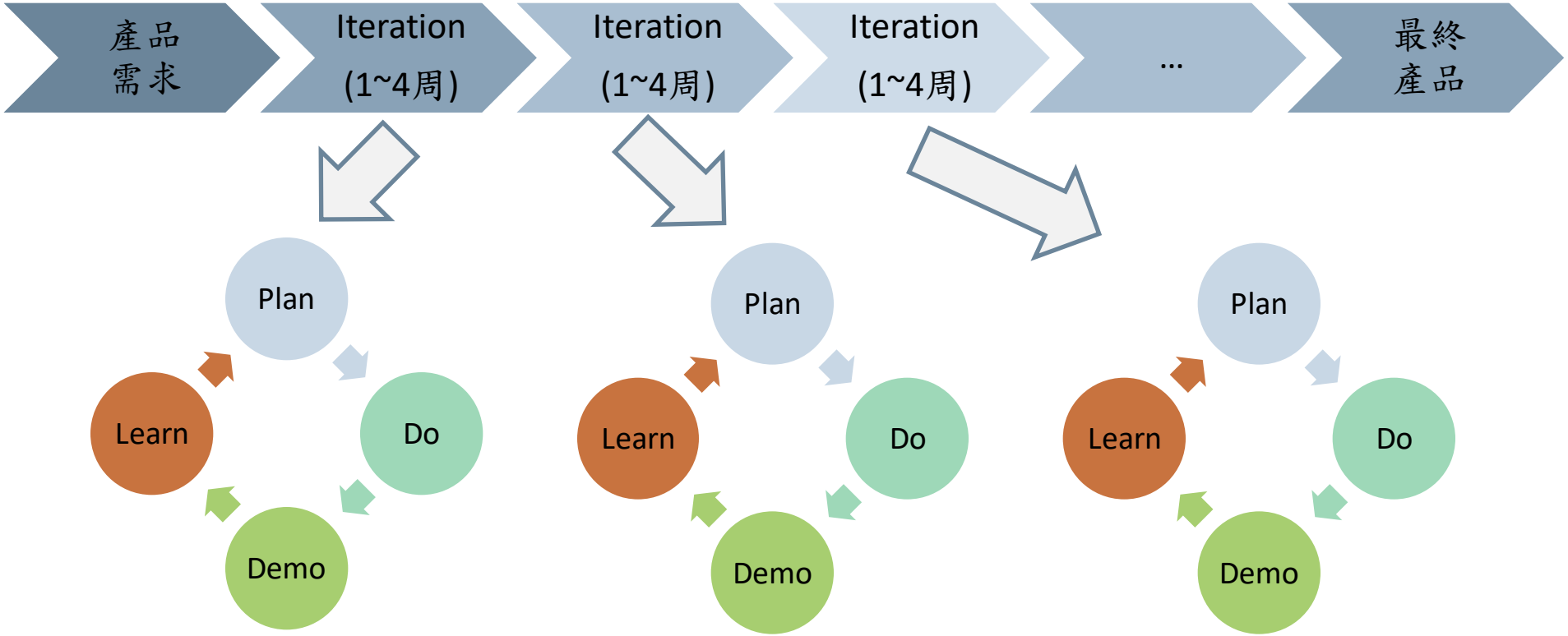
Scrum流程1

- 開發時程，每次衝刺(Sprint)為固定期間(1~4週)。
  - 只須一個團隊、一個開始日期、一個衝刺長度即可開始。
  - 不須完整產品需求的規劃。



Agile Project Management: Iteration

Scrum流程2


產品需求
Iteration (1~4周)
Iteration (1~4周)
Iteration (1~4周)
...
最終產品
Plan
Do
Demo
Learn
Plan
Do
Demo
Learn
Plan
Do
Demo
Learn

Scrum流程3

- Scrum參與流程的三種accountabilities (合稱為Scrum Team)
  - Scrum Master (SM)：協助團隊自主發揮功能、為團隊排除障礙。
  - Product Owner (產品負責人、PO)：定義Product Goal與實現Goal的產品特性，包括功能與品質需求。
  - Developer (開發者)：根據需求進行專案開發。
- Scrum Team是跨職能(cross-functional)的，成員們擁有在 Sprint 內創造價值所需的所有技能。
- Scrum Team是自我管理(self-managing)的，成員在團隊內部決定誰做什麼、何時做以及如何做。

Scrum三支柱[1]

- 透明性(Transparency)
  - 流程和工作必須對執行工作和接受工作成果的人員都是可見的。
    - 實現方式上，在團隊內應讓需求、計畫內容、工作進度、工作產出等都能公開存取。
    - GitHub、Discord、Slack、Trello、Google Drive等工具都可輔助透明性的實現。
  - 透明性促成檢視性。
  - 没有透明性的檢視會產生誤導和浪費。



https://commons.wikimedia.org/wiki/File:Pictofigo-Scrum.png

Scrum三支柱2

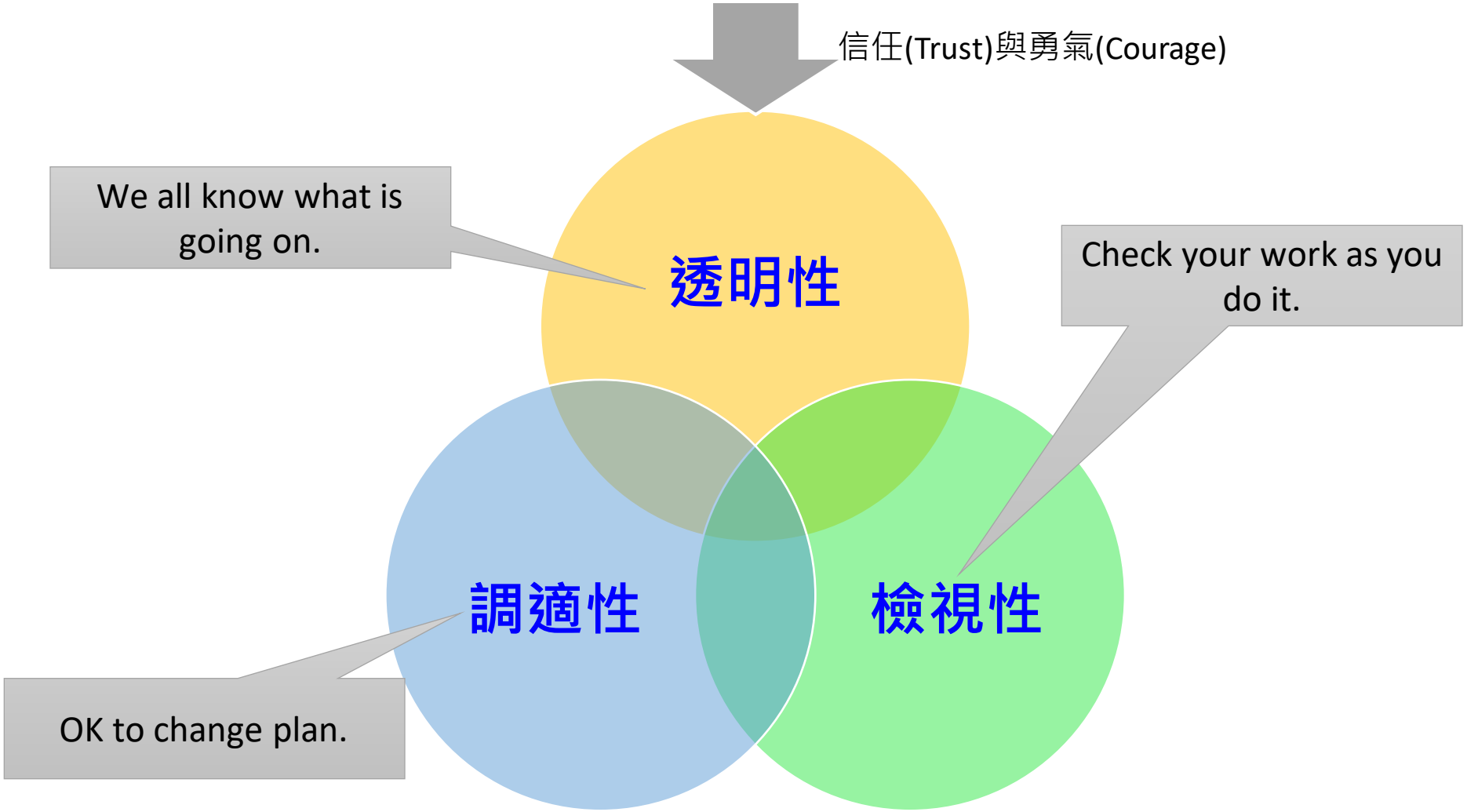
- 檢視性(Inspection)
  - 必須要經常用心地檢視產出物與一致同意的目標進度，以便發現潛在的誤差和問題。
    - 簡單來說，團隊必須要看目前團隊的進展是否與預期符合，了解偏差情況。
  - 檢視性促成調適性。



Scrum三支柱3

- 調適性(Adaptation)
  - 若是流程的任何方面已偏離且超出可接受的範圍，或是所得的產品無法被接受，則必須將所採用的流程或其產物進行調整。
    - 簡單來說，若原本計畫已不吻合現實，要盡快調整。
  - 調整工作必須盡快執行，以減少未來更多的偏差。

Scrum三支柱4


信任(Trust)與勇氣(Courage)
We all know what is going on.
透明性
Check your work as you do it.
調適性
檢視性
OK to change plan.

Scrum產出1

- 產品待辦清單(Product Backlog)
  - Product Backlog Item (PBI)
    - 系統的待處理事項。
    - 可以故事(Story)呈現需求，或任何其他的欲完成工作。
    - 是展示交付給Product Owner的最小單位。
  - 由Product Owner準備，並依市場需求、上市時程、團隊資源、功能重要性等決定優先序。
  - 用以達成Product Goal。

Scrum產出2

- 工作待辦清單(Sprint Backlog)
  - 從Product Backlog選取的PBI。
  - 可將PBI再進行規劃(Planning)，可分解成工作任務(過去亦稱為SBI, Sprint Backlog Item)。
  - 由Developer領取，並在該Sprint中完成。
  - 用以達成Sprint Goal。

SPRINT [NUMBER] BACKLOG

ID	User Story	Tasks	Owner	Status	Estimated effort	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6
US001	As a call centre agent I need to be able to see the caller's previous tickets in their contact record.	Install standard contact record module	J Smith	Completed	16	0.5	0	0	0	0	0
		Demo standard module and identify revised configuration.	J Smith	Completed		0	8	0	0	0	0
		Complete custom configuration as agreed in the demo.	J Smith	Completed		0	0	6	2	0	0
US002	A a customer I need to be able to login to my account from any page on the website.	Install login widget on website.	F Dole	Completed	48	0	0	5	7	2	8
		Apply customer branding.	J Smith	Completed		0	0	0	0	0	0

https://www.stakeholdermap.com/agile-templates/sprint-backlog-template.html#google_vignette

Scrum產出3

- 增量(Increment)
  - 過去稱為潛在可發布增量(Potentially Releasable Increment, PRI)。
  - 增量是在Sprint 所增加的軟體功能，是可以隨時交付給客戶的軟體版本。
  - 應是"可用的(Usable)"的版本。
  - 每一次增量都是基於上一次的增量進行擴增的。

Scrum產出4

- 完成之定義 (DoD, Definition of Done)
  - DoD用以定義Increment符合品質要求的正式描述。
  - Sprint成果符合DoD才能被認定為Increment。
  - DoD範例：
    - 已執行 Code Review，至少一位其他團隊成員已確認無重大問題。
    - 所有與新功能相關的程式碼已被合併至主分支，且無合併衝突。
    - 功能已在測試環境部署，並成功通過E2E測試。
    - API 回應時間不超過 200 毫秒。
    - 相關的技術文件(設計文件)、API 規格書已更新。

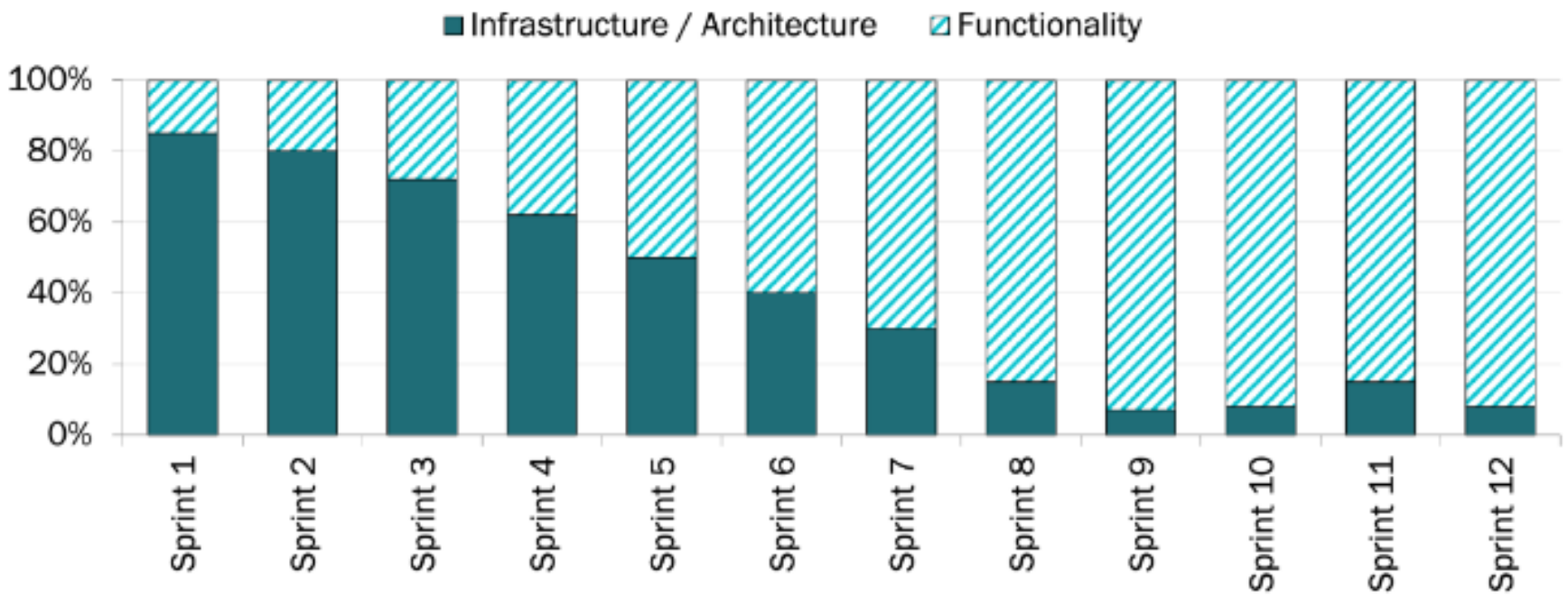
使用者故事(User Story)

- Product Backlog Item (PBI)可以User Story表示
- 每個User Story包含：
  - 需求的簡要描述。
    - As a (user type), I want to (goal), so that (reason)…
  - 以流程步驟或對話形式，描述操作程序以及所得的效益。
  - 也會描述如何測試或展示，確認系統符合Product Owner期望。

Scrum事件(Event)

- Sprint是所有其他事件的容器。
  - 這些事件可視為儀式(ceremony)。
- Scrum中的每個事件都是用來檢視與調適Scrum artifacts 的正式機會。
  - 這些事件包含Sprint Planning、Daily Scrum、Sprint Review、Sprint Retrospective。
- 最理想的是，所有事件都在同一時間同一地點舉行，以減少複雜性。

底層架構與功能的規劃比例

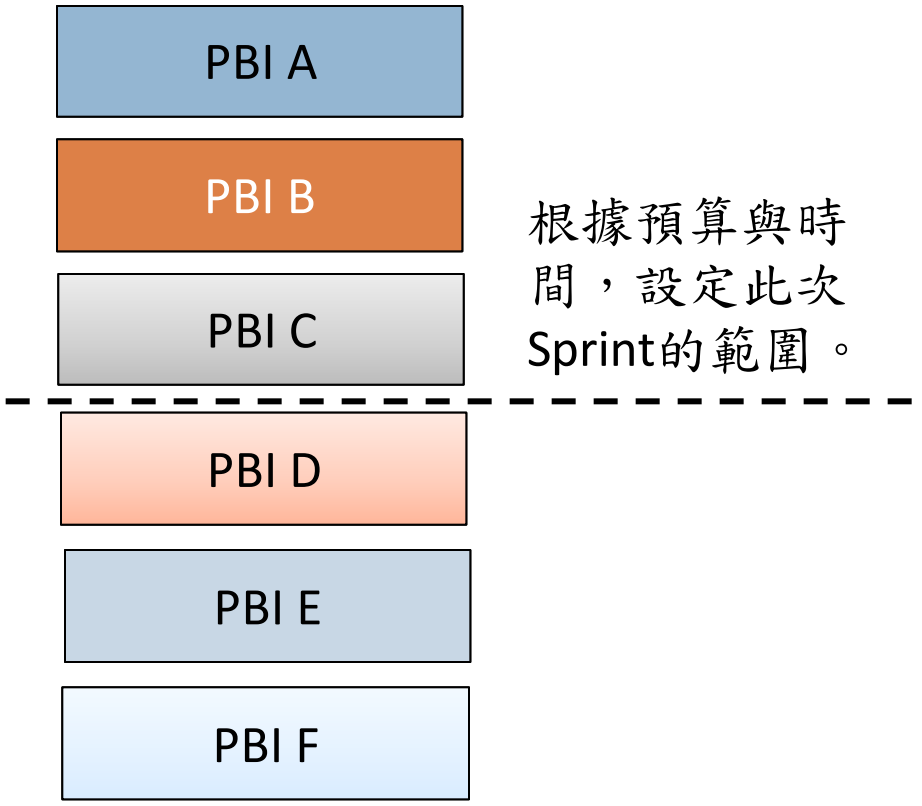


Scrum需開的四種會議[1]

- 第1種會議: Sprint Planning (衝刺規劃會議)
  - Sprint Planning 討論以下主題：
    - 主題一：為什麼這次 Sprint 有價值？
    - 主題二：這次 Sprint 能完成(Done)什麼？
    - 主題三：如何完成所挑選的工作？
  - 每一衝刺前，Product Owner帶領整個團隊開會，自Product Backlog中**選擇**若干Product Backlog Item (PBI)。
    - 在那之前有可能需要進行Backlog Refinement
  - 再由Scrum團隊將PBI**分割**成數個工作任務(task)，紀錄於Sprint Backlog，成為Sprint Backlog Item (SBI) 。
    - 整個團隊共同決定工作任務之分配。
  - Sprint Planning 是有時間盒限定(timeboxed)的，一個月的 Sprint 最多為8個小時

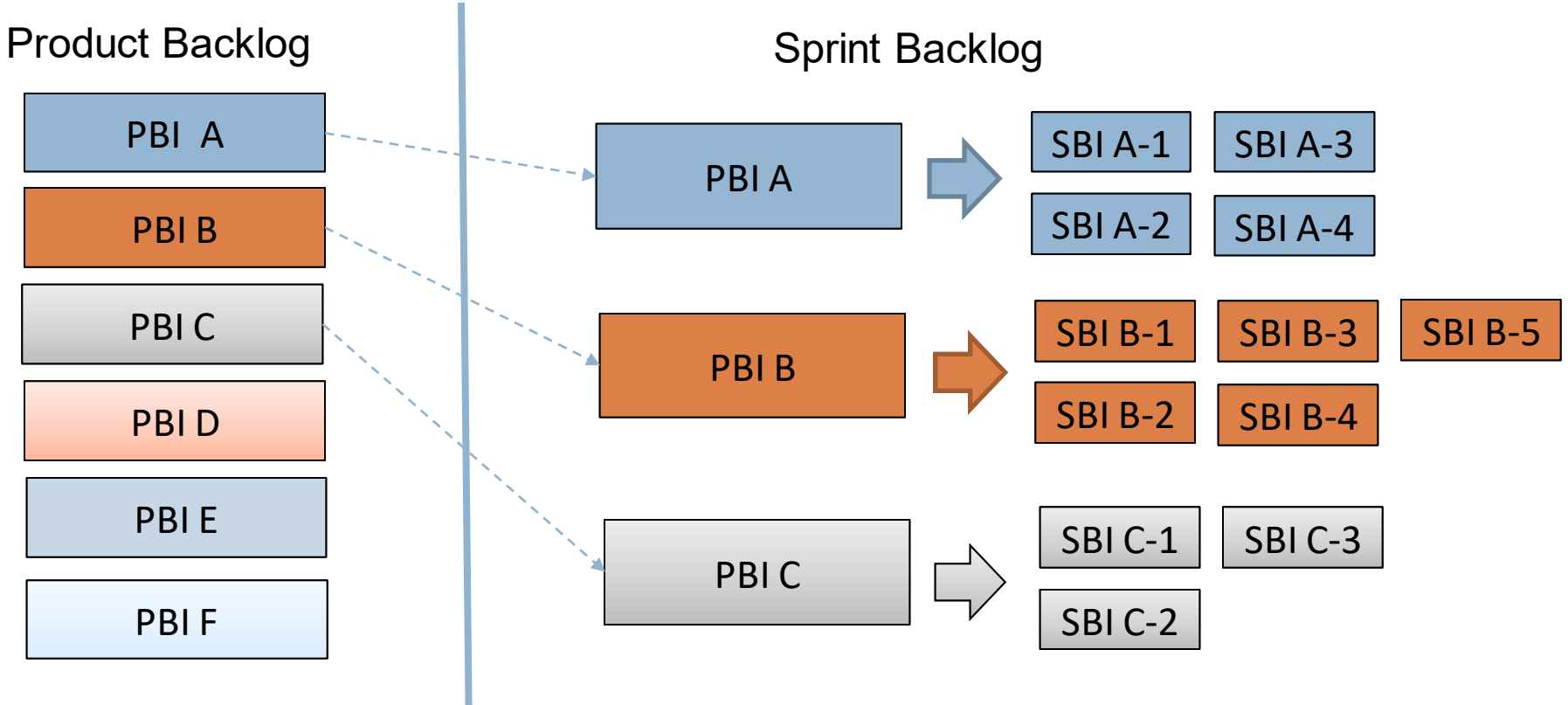
Scrum需開的四種會議2

- 如何挑選每次Sprint的PBI?
  - 我們應該依照PBI的價值(預估獲利)與成本(預估花費時間)，決定PBI的優先順序，以決定每個Sprint應挑選哪些PBI，並拆解為SBI。



Scrum需開的四種會議3

- 挑選PBI (可能是使用者故事)、切割PBI為任務(過去也稱為SBI, Spring Backlog Item)
  - 任務包含分析、設計、實作、測試等各種形式的工作。



Scrum需開的四種會議4

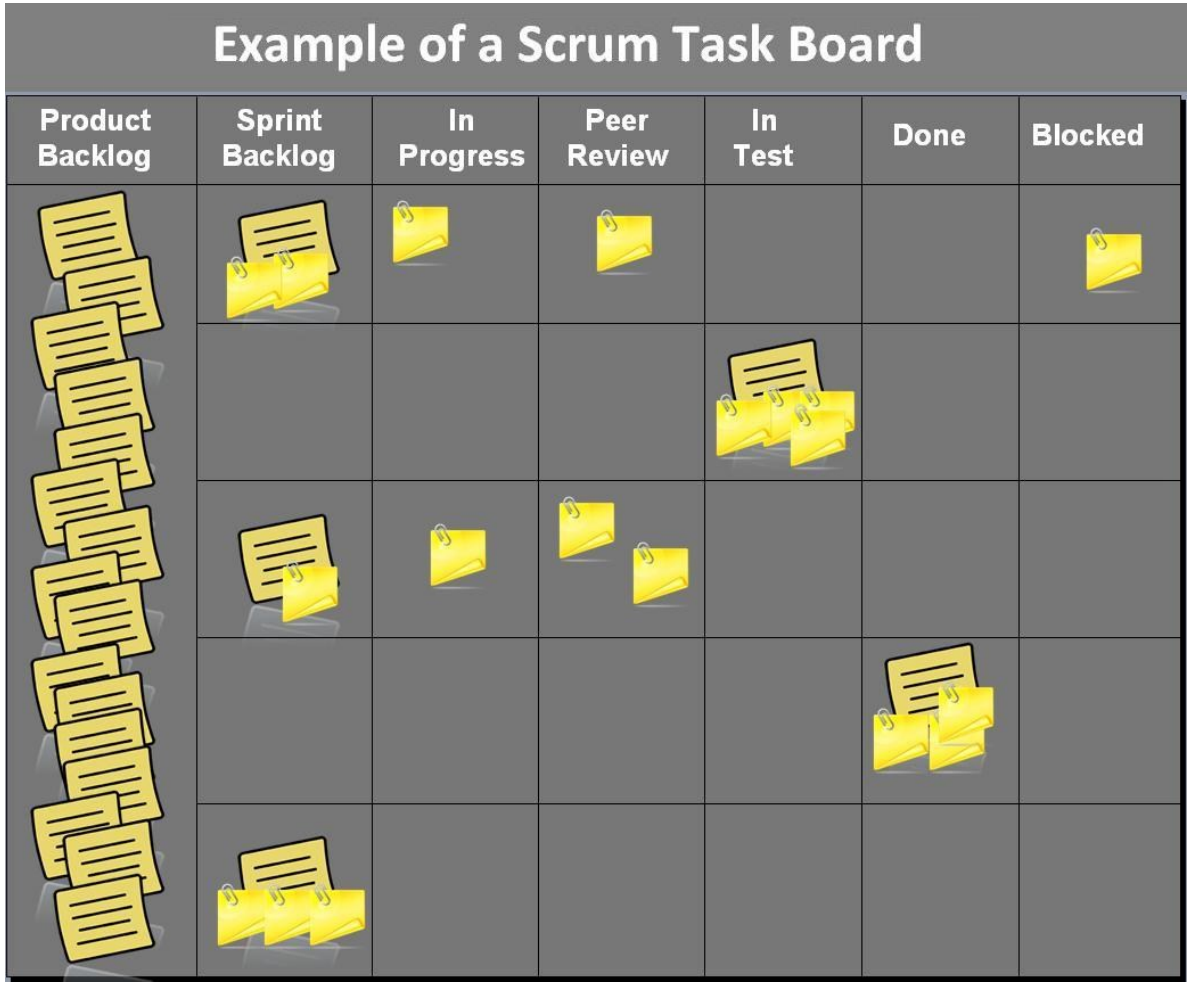
- Sprint Backlog範例 (會議排程系統)

PBI	Backlog Item	Task	
1	身為一個使用者，我可以觀看所有會議之列表。	1-1	會議資料表設計與設定
		1-2	會議模組設計與實作
		1-3	會議列表GUI之設計與實作
		1-4	測試案例撰寫與執行
2	身為一個使用者，我可以瀏覽會議行事曆行事曆中包含我發起之會議或受邀請之會議。	2-1	設計與實作Calendar GUI
		2-2	整合DatePicker Widget元件
		2-3	整合Google Calendar API
		2-4	整合會議模組與Calendar GUI
		2-5	測試案例撰寫與執行
3	身為一個使用者，我可以在系統上發起一項會議。	3-1	發起會議GUI之設計與實作
		3-2	整合DAO與Initiation GUI
		3-3	測試案例撰寫與執行

Scrum需開的四種會議5

- 第2種會議: Daily Scrum
  - 之前也叫Stand-up Meeting
  - 每個上班日定時召開(如上午9:30)，應少於15分鐘。
  - 成員針對目前進展進行檢視，並進行必要的調適。
  - 可討論問題：昨天完成甚麼？今天準備完成甚麼？遭遇問題？
  - Scrum board可於此會議時管理(平時即可管理)
    - 成員在看板修改工作狀態(完成花費時間、日期)，將其移至已完成區(Done)。
      - 預期每次可聽到1~2個Done。
    - 若PBI底下的SBI均完成，將此PBI也移至Done。
    - 確定(認領)今天準備做的工作，自待領區(Backlog)取工作，標上姓名代號與領取日期，移至領出區(In Progress)。

Scrum需開的四種會議6



Scrum經常搭配任務看板 (Task Board，可以用Trello實現)

任務看板是Scrum達成可視化(Visibility)的重要方法之一

https://commons.wikimedia.org/wiki/File:Scrum_task_board_example.jpg

Scrum需開的四種會議7

- 第3種會議: Sprint Review
  - 在當次衝刺即將結束前，由團隊與相關人士(Stakeholder)對此次Sprint所完成之增量進行討論
    - 可展示已完成增量之功能及品質特性。
    - 可擬定產品後續發展方向。
  - 一個月的Sprint ， Sprint Review最多為4個小時。



https://commons.wikimedia.org/wiki/File:BIS-Sprint-Final-24-06-13-01.jpg

Scrum需開的四種會議8

- 第4種會議: Sprint Retrospective
  - 每一衝刺最後一項活動為回顧，通常由Scrum Master主持。
  - Scrum Team 討論此次 Sprint 中，哪些進展順利、遇到哪些問題、如何(或為何無法)解決這些問題。
  - 接下來再進行下一個Sprint，如此週而復始。
  - 一個月的Sprint，Sprint Retrospective最多為3個小時。



https://commons.wikimedia.org/wiki/File:Team_Retrospective.jpg

Scrum角色1

- Scrum由一位Scrum Master來營造開發環境：
  - 1. 由Product Owner將錯綜複雜(complex)問題所需之工作事項整理排列成為一份Product Backlog；
  - 2. Scrum Team 在一個 Sprint 中將所挑選的工作轉化為價值的 Increment；
  - 3. Scrum Team 和利害關係人(stakeholders)檢視結果，並為下一個 Sprint 進行調整；
  - 4. 重複上述步驟。
- Scrum團隊通常10人以下。

Scrum角色2

- Product Owner (產品負責人、PO)
  - 內部或外部的客戶、委託者或聯絡人代表，只會有一個人。
  - 開發並明確的描述與溝通 Product Goal。
  - 負責產出與排序產品待辦清單(可由其他人協助)，
  - 根據ROI (Return On Investment)決定PBI優先次序。
    - 根據市場價值，為每個Sprint調整產品功能及優先順序。
    - 對產品能否為公司帶來利益負責。
  - 根據目標監控產品進展。
  - 認可Sprint Demo的結果。
  - PO是產品(Product)的守護者，決定“What”。

Scrum角色3

- Developer
  - 跨職能(可分析、設計、寫程式、測試)。
  - 透過程序改善，創造團隊基本行為規則。
  - 打造Sprint 的計畫。
  - 每天調適其邁向Sprint Goal的計畫。
  - 負責完成工作任務、建構產品增量，並於Sprint結束前展示工作成果。
  - 通常只能在前一個Sprint結束後與下一個Sprint開始前才能更換成員。
  - Developer是產品的開發者，決定“How”。

Scrum角色4

- Scrum Master (SM)
  - Scrum Master負責按照 Scrum 指南來建立Scrum環境，方式是透過幫助Scrum Team(與組織內部的每個人)了解 Scrum的理論與實作。
  - 確保所有的Scrum事件都舉行，有建設性、有成效的並且保持在時間盒(timebox)內進行。
  - 確保Developer能全力投入，保護團隊免於不必要的干擾、移除障礙。
  - 確認需求，增加生產力。
  - 成為Product Owner和Developer間的橋梁，推動雙方合作。

Scrum角色5

- Scrum Master的特質
  - 教練、領導者、協調者、照顧者、牧羊人、促進者、指導者、激勵者等。
  - 不是專案經理(Project Manager)，而是為團隊做事。
  - 開闊心胸願意接受改變、能處理衝突。

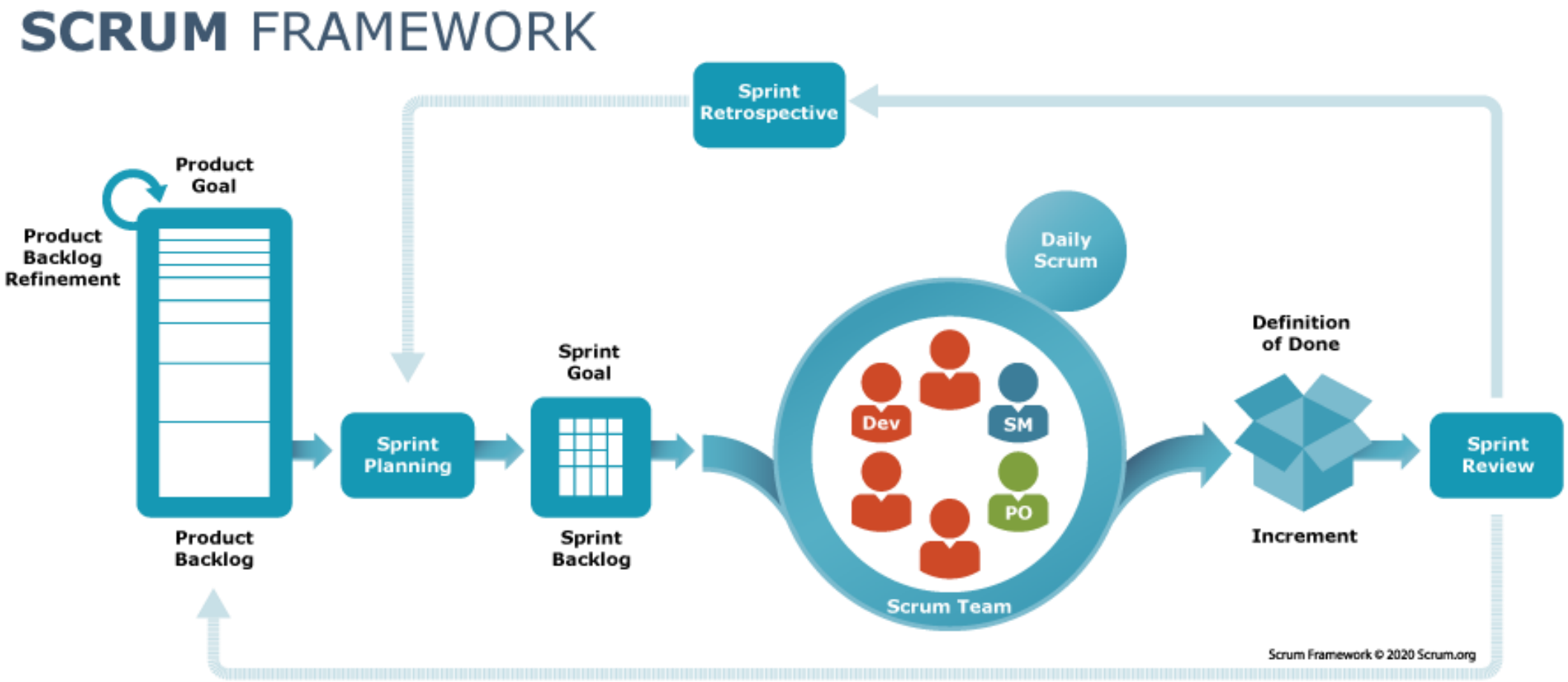
Scrum的價值觀



https://sammanipalansuriya.medium.com/happy-architecture-creating-a-culture-of-happiness-by-building-excellence-in-to-your-products-8eae3695043c

- 承諾：大家承諾一起完成目標，並彼此相互支援。
- 專注：大家專注於現階段目標，盡可能地朝向目標前進。
- 開放：大家對於要執行的工作及遭遇問題抱持著開放的態度。
- 尊重：大家互相尊重對方是個有能力和獨立的人。
- 勇氣：大家有勇氣去做對的事情、處理棘手的問題。

Scrum總結



https://www.scrum.org/resources/scrum-framework-poster

Scrum.org The Home of Scrum

總結

- Scrum框架
  - 創造公開、可視化的工作環境。
  - 創造自組織的工作模式。
  - 透過價值(Value)導向的規劃方式提升專案價值。
  - 讓團隊成員可迭代式(Iterative)的完成專案願景。

https://www.youtube.com/watch?v=XU0llRltyFM (10 分鐘介紹 Scrum)

補充資料：極限製程

極限製程(eXtreming Programming, XP)[1]

- 極限製程(XP) 是Kent Beck 於1999 年提出， 目的是提倡更能「擁抱改變」(Embrace Changes)的敏捷開發方式(Agile Method)。
  - 極限製程的「極限」有著極端的含意，因為它提出許多極端的做法，例如，極端地倡導多回合開發方式、極端地要求顧客參與、極端地強調測試的重要性等。

極限製程之特色1

- 小幅釋出(Small Releases)
  - 在短時程的開發週期內(一至三週)可以釋出具有完整功能的版本，並持續改善。
- 規劃遊戲(Planning Game)：
  - 開發者與客戶共同合作進行每個開發週期的規劃與預估。
- 測試(Testing)
  - 開發者負責撰寫自動化的單元測試案例(Unit Test Case) ，客戶則負責撰寫驗收測試案例(Acceptance Test Case)。
- 客戶駐點(OnSite Customer)
  - 顧客代表也是開發團隊成員之一，在開發過程中必須全職參與開發團隊的討論，如此可以省去需求文件化的時間，以及避免閱讀需求文件可能產生的錯誤。

極限製程之特色2

- 重構(Refactoring)
  - 在不改變程式行為介面的前提下（即可接受的輸入與可產生的輸出不變），透過調整程式碼結構或改寫程式碼來達成改善程式碼品質的目標。
- 團隊共有程式碼(Collective Code Ownership)
  - 程式碼是由團隊中的所有人共有，而非每個人僅負責系統的一部份。換言之，團隊中的任兩人應可隨時交換其目前撰寫的程式碼，並能順利地繼續完成開發。
- 配對程式開發(Pair Programming)
  - 兩位程式開發者一起使用同一個鍵盤與同一個螢幕，共同開發程式。其模式為一人主寫，另一人則進行即時的審查(review)，透過兩人不同的觀點盡早找出程式的缺陷，以改善程式的品質。

極限製程之特色3

- 程式碼標準(Coding Standard)
  - 團隊應統一規範所有成員撰寫程式碼的風格，包含變數命名、縮排、註解形式等。倘若團隊中每個人的程式風格均不相同，容易造成日後整合與維護的困難。
- 簡單設計(Simple Design)
  - 應盡可能用最簡單的設計達成使用者需求。XP強調設計並不是一次就可以到達完美，透過簡單的設計、測試與設計的改善，逐步修正設計，使系統可以切合使用者需求與系統品質。
- 隱喻(Metaphor)
  - 提供方便簡單的隱喻，以便進行系統行為的構思與討論。
  - 舉例而言：透過暫存區(buffer)方式提升字元輸出效率的設計，即可命名為垃圾車(收集足夠的字元才一次輸出)。

極限製程之特色4

- 可持續的速度(Sustainable Pace)
  - 以可持續的速度進行工作（通常是指一週最多工作40小時），以建構有品質的程式碼。
- 持續整合(Continuous Integration)
  - 團隊中多位開發者之程式碼應能持續進行整合，其整合週期應至少一天一次，以確保隨時均有可用的最新版本可供測試與展示。

極限製程時程規劃範例

- 極限製程開發時程規劃範例

	1	2	3	4	5	6	7	8	9	10	11	12
回合一／規劃	2w											
回合一／設計測試案例	2w											
回合一／設計與實作		2w										
回合一／執行測試與整合		2w										
回合二／規劃			2w									
回合二／設計測試案例			2w									
回合二／設計與實作				2w								
回合二／執行測試與整合				2w								
…												
回合六／規劃											2w	
回合六／設計測試案例											2w	
回合六／設計與實作												2w
回合六／執行測試與整合												2w

極限製程的優缺點

- 極限製程開發方式的優點：
  - 較敏捷的開發方式，有快速的回饋機制來修正過程中產生的各項衝突或錯誤。
- 極限製程開發方式的限制：
  - XP 的各項原則都有互補作用，一旦某項原則沒有確實做好，就有可能帶來危機。
    - 若測試程式撰寫不完整，系統便無法靠著通過測試來達到頻繁改版的穩定性。
    - XP所強調的是程式，而非文件，若程式重構得不夠，使得設計不夠簡單，便有可能會造成日後維護的困擾。
  - 實施上若沒有確實落實，容易流為「沒有流程」，造成流程的失效。

Any Question?

